A New ROS-based Hybrid Architecture for Heterogeneous Multi-Robot Systems

Chunxu Hu, Can Hu, Dingxin He, Qiang Gu

Department of Control Science and Engineering, Huazhong University of Science & Technology, Wuhan 430074
E-mail: huchunxu@hust.edu.cn

Abstract: In this paper, a new hybrid architecture for heterogeneous multi-robot systems is presented. Personal computers as central server and embedded systems as nodes are integrated in this architecture based on robot operating system. The complex computation and visualization are processed on server, while real-time tasks are completed in robot nodes with embedded systems. Compared with the existing schemes, this hybrid architecture can keep a good balance between flexibility, expansibility, power consumption and real-time capability, without causing degradation in performance. According to the architecture, a low-cost and high-performance robot node named hybrid real-time mobile robot platform is implemented on system-on-chip. The experimental result of multi-robot following verifies the feasibility and usability of the proposed scheme.

Key Words: robot operating system, multi-robot systems, hybrid architecture, hybrid real-time mobile robot platform

1 INTRODUCTION

During the last few years, the field of multi-robot systems (MRS) has received increased attention because of its superiority. Compared with single robot systems, MRS have incomparable advantages, such as (1) reducing complexity and cost of the single robot in the design process, (2) enhancing efficiency by means of task decomposition, (3) increasing the redundancy and robustness through mutual cooperation, (4) improving the accuracy through information exchange among several robots [1]. Therefore, MRS have produced a potentially wide set of applications in search and rescue, military missions, industrial production, space exploration, social services, etc.

MRS can be divided into homogeneous MRS and heterogeneous MRS. Since homogeneous MRS are generally equipped with the same (or similar) types of sensors and actuators, which almost have the same function and application area, so they can only complete limited tasks. On the other hand, heterogeneous MRS can make up for this shortcoming with different sensing, moving and computing capabilities. Thanks to various sensors and actuators, each member in systems can be designed to complete a specific task based on its special expertise ability. In the aspect of robot control, embedded systems have been widely used for reducing power consumption and production cost. But compared with personal computers (PCs), embedded systems are short of computing capability.

With the rapid growing scale and scope of robotic systems, the robot development becomes increasingly complex and sophisticated. Aiming at this point, it is necessary for developers to have a good knowledge of various fields, and be familiar with different kinds of development environments. Moreover, the existing software modules are quite difficult to be reused on another platform, because there are no software standards for various robot hardware and drives [2]. To meet these challenges, Willow Garage releases an open-source distributed framework for robot software development named robot operating system (ROS), starting an upsurge of learning and researching ROS [3, 4, 5].

ROS is not an operating system in traditional view of processing and scheduling. It can provide a standard structured communication layer based on host operating systems of heterogenous compute cluster. Thanks to a strong support of the ROS community which is maintained by open-source ROS developers, the number of software packages and potential collaborators has been growing exponentially [6]. Since its high code reusability and extensibility, robot systems based on ROS can be used for secondary development and complex application conveniently by leveraging the work of others in the community [7]. Nevertheless, there remain some deficiencies of ROS until now.

- Most function packages are designed for PCs. If they can be applied into embedded systems, the robot cost and power consumption will be better controlled.
- The ROS function packages do not have determinate processing time because ROS is a non-real-time system.
- For ordinary developers, ROS-based robots such as Pioneer Robot, are often too expensive for MRS. Even the sensors such as SICK laser are costly.
- Even though ROS is a distributed framework, software packages are few for MRS applications in the community.

In order to explore advantages of MRS and ROS, this study designs a new ROS-based hybrid architecture for heterogeneous MRS which combines PCs and embedded systems together. In this way, flexibility, expansibility, power consumption and real-time capability of the MRS can maintain a good balance without causing degradation in performance.

978-1-4799-7016-2/15/$31.00 ©2015 IEEE

This paper is organized as follows. Compared with the existing architectures and traditional ROS scheme, the new hybrid architecture is discussed in section II. Section III presents a low-cost and high-performance robot node named hybrid real-time mobile robot platform based on this architecture. The heterogeneous multi-robot experimental result is shown in section IV. Section V is devoted to some conclusions and future work.

2 DESCRIPTION OF THE ARCHITECTURE

The architecture of MRS defines the topological structure and function allocation among the robots in the system. Thus, designing an appropriate architecture is an important prerequisite for completing multi-robot tasks successfully.

2.1 Existing Architectures of MRS

The main architectures of MRS are centralized, distributed and hybrid [8].

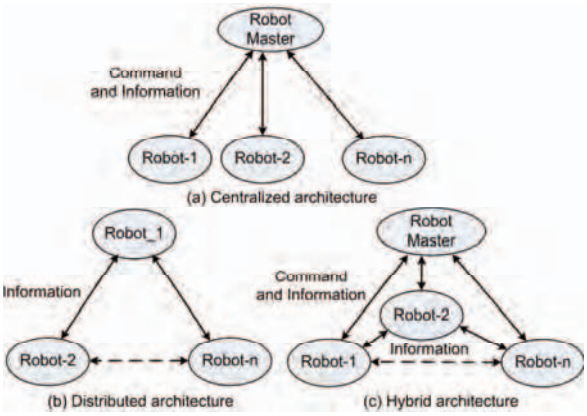

Fig 1. Existing architectures of multi-robot systems.

1) Centralized architecture

In this architecture shown in Figure 1(a), there is always a more powerful robot as the group master that controls all the movements and information of other robots. Therefore, most robots of the system are executive bodies without any ability of independent movement and coordination. And the failure of the robot master will directly lead to a system crash.

2) Distributed architecture

The distributed architecture (Figure 1(b)) allows each robot to choose its own action autonomously. Each robot of the system coordinates its behaviors through the exchange of information with others. Unfortunately, this structure requires high communication efficiency, and lacks the global performance and predictability.

3) Hybrid architecture

This architecture (Figure 1(c)) combines the advantages of the centralized architecture and the distributed architecture. In this way, the robot master can have full information of the system and send overall resolutions. Every robot of MRS can complete the task independently and communicate with not only the robot master but also the other robots. Thus, it is suitable for complex and dynamic environment.

2.2 Traditional ROS Scheme

In the traditional ROS scheme shown in Figure 2, robot sensors and actuators exist as nodes, which are distributed on ROS-based robots. Communication messages between nodes are always processed and transmitted by PCs in robot. But the PCs can't connect directly with various hardware devices because of the inconsistent interfaces. As a result, driver boards are absolutely necessary between PCs and devices, which are connected with PCs through USB and serial port, and connected with devices through I/O, SPI, I2C, etc. In some scenarios, it is a good choice to add a microcontroller board such as Arduino [9] or Raspberry Pi [10] to manage drivers and peripherals.

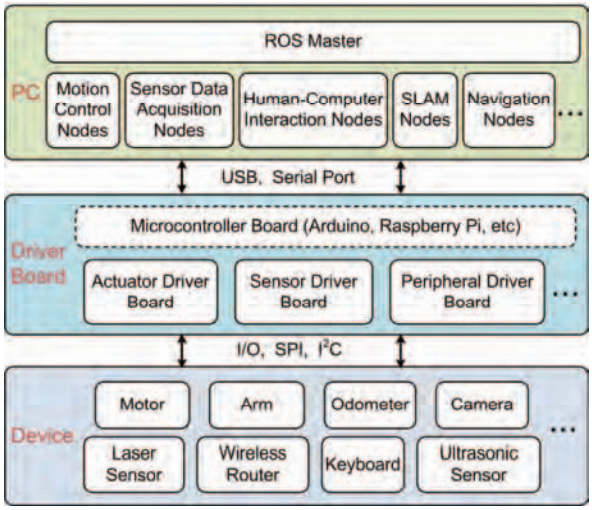

Fig 2. Traditional robot operating system scheme.

This scheme is convenient and universal, but not very feasible and flexible for MRS. Firstly, since the unique ROS master is required to manage all nodes in the system, there must be a robot master in multi-robot cooperative applications. So it has the same shortcomings as the existing centralized architecture. Secondly PC-based solution is less flexible than embedded system solution. Thirdly, it increases cost and limits resources recycle because of the nonuniform and complicated hardware structures in MRS.

2.3 New ROS-based Hybrid Architecture

As we know, the characteristics of embedded systems are small size, low power consumption, low cost, high flexibility, etc. With the rapid development of a multi-core system, using the multi-core processor has become an effective way to obtain computational ability. At the same time, the progress of electronic technology makes field programmable gate array (FPGA) become widely used in embedded systems. FPGA can greatly improve the hardware flexibility of embedded systems, and provide a reliable guarantee for real-time computation. Some high-end devices are already available to integrate multiple processors and FPGA in a single system-on-chip (SoC) platform. If the characteristics of ROS and that of embedded systems are combined, the solution can improve the flexibility and universality greatly for MRS.

Considering the existing architectures, a new hybrid architecture is described in Figure 3(a). Different from the

existing hybrid architecture, the robot master is replaced by a PC-based server. The server is ROS master in the hybrid architecture for providing powerful computing and image processing capabilities. Embedded mobile robots are configured as nodes, which are different from sensor nodes or actuator nodes in the traditional scheme shown in Figure 3(b). Because it is not easy to manage a large number of nodes in complex MRS. Multi-robot coordinated and controlled by the server can work together to enhance system efficiency and performance with a wireless network through ROS. And every robot node can join or quit the network at any moment.

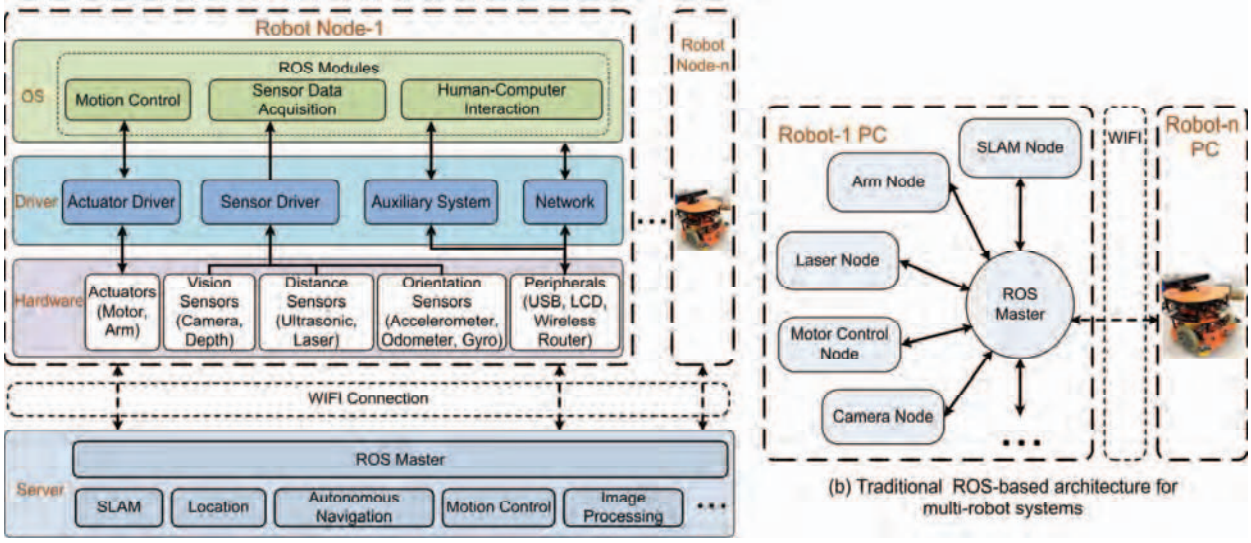

Fig 3. Comparison of new and traditional ROS-based multi-robot systems architectures.

1) Robot Node

The robot node which can be configured conveniently is composed of three-layer structure containing hardware layer, driver layer and operating system (OS) layer.

The hardware layer is the foundation of the robot. Actuators, sensors and peripherals are important for a universal robot hardware platform. The actuators include motors and arms that assist robot to complete the movement and action. The sensors can be divided into three types according to their functions. The vision sensors such as camera and depth sensor can detect three-dimensional information about the real environment for upper computer (server). Ultrasonic and laser sensors are chosen as the distance sensors to improve accuracy of the environment information. The third type sensor is orientation sensors, such as the accelerometer, odometer and gyro, to detect position and angle information of the robot. The wireless router is a basic peripheral equipment to provide communication media between robot nodes and the server.

The driver layer is used to provide information channels between OS layer and hardware layer. All sensors and actuators in robots should be initialized in this layer when the system starts up. In addition, the drivers provide the application programming interfaces to the operating system, reducing the coupling between system hardware and software. The auxiliary system is used to monitor system, add auxiliary functions and expand peripheral drivers. The network module drives hardware network card and processes upper data into TCP/IP packets, providing a network environment for the system.

In the OS layer, Linux is adopted as the embedded operating system. The ROS core library which has only 10-plus MB code scale is transplanted into Linux for providing interfaces of robotics middleware. The sensor information from the previous layers is converted to the ROS standard data format and transmitted to the corresponding modules. Motion control is a closed-loop module based on the sensor data and the server commands. The single robot node has a certain autonomous ability to finish simple tasks without the server control. Furthermore, human-computer interaction is designed in order to develop and monitor system conveniently. All the information in this layer is sent to the server through the wireless network.

2) Server

The server as ROS master is the core unit to deal with robot complex functions through collecting various robot data. Compared with the existing hybrid architecture, the server takes over the work of the robot master to control all movements and information in MRS. Furthermore, the PC-based server can reduce the design difficulty of the robot master and guarantee the whole system operational capability. In the case of complex and dynamic environment, the server can be configured easier than the robot master, because PCs have more excellent and convenient human-computer interaction. And it can also be a huge computer cluster to obtain higher stability, redundancy and performance for MRS with massively parallel processing.

After processing the information from robot nodes, the server sends control commands to corresponding robot nodes. Meanwhile, three-dimensional visualization environment model and robot real-time status can be visualized on the server. ROS contains a lot of useful function packages from the open-source community. Users can quickly develop desired functions with these packages, such as simultaneous localization and mapping (SLAM), autonomous navigation and image processing [11].

In this new hybrid architecture, robots still maintain original distributed architecture. But each robot can have different hardware structures and functions. Benefit from the ROS server, the MRS becomes more controllable, configurable and powerful. According to the specific applications, robots for missions and PCs for calculation can be easily integrated into heterogeneous MRS. And low-cost embedded systems reduce the number of PCs and driver boards for complex MRS. So the increasing number of robot nodes does not lead to a sharp increase of system cost.

3 IMPLEMENTATION EXAMPLE

According to the previously considered architecture for MRS, this section presents a hybrid real-time mobile robot platform (HRMRP), mainly for indoor service [12]. It is suitable to play the part of low-cost and high-performance robot nodes for MRS.

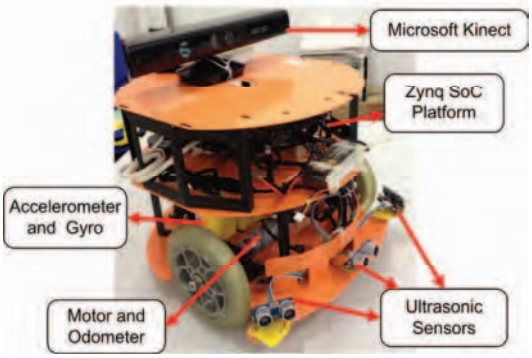

Fig 4. Hybrid real-time mobile robot platform.

The HRMRP (Figure 4) is composed of two driving wheels and one steering wheel attached to the main frame. And the top speed is 83 r/min. The body containing three layers is made of aluminum alloy. The dimensions of the robot are 316 mm × 313 mm × 342 mm (H × W × L, respectively) with a total mass of 5 kg.

The new hybrid architecture for heterogeneous MRS described in section II can provide a good support for not only the HRMRP but also other robots nodes with different structures, shown in Figure 5.

3.1 New Hybrid Architecture Implementation

1) Robot Node

The hardware is realized on Xilinx new generation hardware platform Zynq-7000 [13], which makes the performance of SoC forward to a higher level. Zynq SoC is composed of processor system (PS) and programmable logic (PL). The PS part is constructed around the ARM Cortex A9 dual core processor, containing common peripherals, such as network, USB and memory controller. The PL is based on Xilinx 7 series programmable logic. A Microsoft Kinect (640 × 480 resolution) is chosen as the core sensor [14]. Compared with traditional laser or camera, the information achieved by Kinect sensor is abundant, including not only three-dimensional depth data, but also color image data with a measurable range of 400 ~ 8000 mm. On the short distance range, it needs several ultrasonic sensors to avoid collision. In order to obtain the position and angle of the robot, odometer (hall sensor, 16 pulses per revolution), gyro (ADXRS453) and accelerometer (ADXL212) are also equipped.

The driver layer is divided into two parts for PL and PS respectively. With programmable hardware, I/O expansion and hardware acceleration for algorithms are the main PL characteristics in this layer. The other drivers in PS part provide interfaces of PL to PS and device drivers for operating system.

In order to take advantage of the ROS communication mechanism, the ROS core library is ported to the embedded Linux operating system (Ubuntu) based on Zynq SoC. It has a 667 MHz processor with 512 MB random access memory and 32GB secure digital memory card. The system development is based on ROS interfaces of robotics middleware with C++. Thanks to the excellent design of ROS, we can conveniently reconfigure parameters online without recompile source codes.

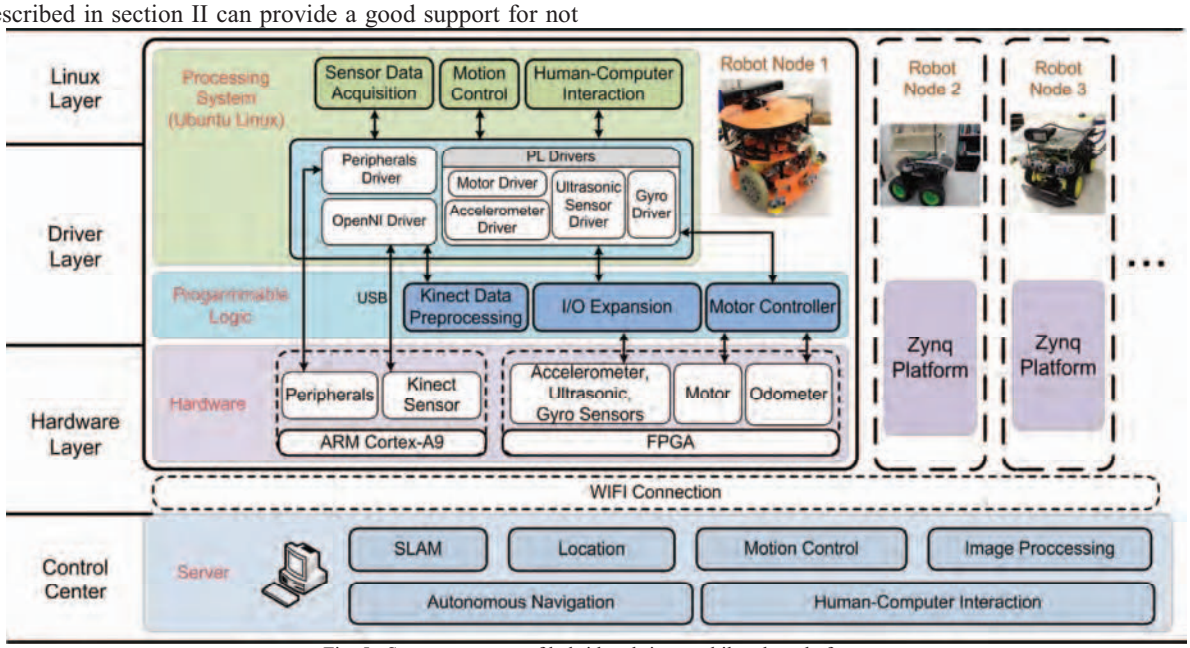

Fig. 5. System structure of hybrid real-time mobile robot platform.

2) Server

A single PC with Linux operating system is chosen to act as the server for general applications. The ROS master on the server is responsible for managing all robot nodes, which can be visualized in a visual component named Robot Visualizer (RViz). The communication between robot nodes and the server is via 802.11n networking in the system. A few common functions have been realized on the server, such as SLAM, location, autonomous navigation, image processing, motion control, etc.

When the HRMRP starts up, the running sequence of system modules is as follows:

- The depth and image data of Kinect collected by Linux layer are preprocessed in PL. At the same time, other sensors data such as ultrasonic data or accelerometer data are also sampled.
- Raw sensors data are fused and converted to inertial measurement unit, position, velocity, image and depth data according to ROS standard data structure.
- The corresponding functions according to the task requirements are processed with the data from HRMRP or other robot nodes. The processing procedure can be seen through RViz.
- The robot nodes can execute server control commands for multi-robot tasks and finish single robot tasks by themselves.

3.2 Comparison with other Robot Platforms

An efficient robot platform with ample hardware and software support is essential for education and research. HRMRP is quite a relatively common platform for most multi-robot and single robot applications. From the most popular intelligent robot platforms, we choose two typical platforms to compare with HRMRP. The result of this comparison is shown in Table 1.

1) TurtleBot-2

TurtleBot-2 provides a basis development platform with multiple sensors for entry-level robotics enthusiasts [15]. Using TurtleBot-2 own hardware and software, the developers can focus on the practical applications. Because the promulgator of TurtleBot-2 is the same as ROS, the abundant software packages in ROS community can be applied easily. Unfortunately, the laptop in TurtleBot-2 limits the interface scalability of the system. At the same time, every robot equipped with a laptop is waste for lightweight multi-robot applications considering the cost.

2) Pioneer 3-DX

Pioneer 3-DX is a small lightweight two-motor differential robot for indoor laboratory or classroom use [16]. With a variety of optional accessories such as laser-range finders, robotic arms and grippers, Pioneer 3-DX can handle various tasks in complex environment. However, laptops are also necessary for the robot system. Even though the function of Pioneer 3-DX robot is abundant, the cost is expensive with these accessories for MRS.

3) HRMRP

Compared with TurtleBot-2 and Pioneer 3-DX, HRMRP has the same dimension and construction, but does better in interface scalability, sensor diversity and cost control. At the same time, the comprehensive performance of the robot is still outstanding. In multi-robot network and applications, HRMRP can be more flexible and efficient than the other two robot platforms. However, they can still work together well to complete multi-robot tasks.

Table1. Comparison of HRMRP and Other Robot Platforms

Item	TurtleBot-2	Pioneer 3-DX	HRMRP
Dimension (mm)	354 x 354 x 420	455 x 381 x 237	316 x 313 x 342
Weight (kg)	6.3	9	5
Construction	Aluminum	Aluminum	Aluminum
Battery (mAh)	2200 (Li-Po)	7200 (Lead-acid)	4000 (Li-Po)
Sensors	Kinect, Odometer, Gyro, Accelerometer	Sonar, Odometer, Lidar	Kinect, Camera, Odometer, Gyro, Ultrasonic, Accelerometer
Ports	USB, Ethernet	USB, RS232, Ethernet	USB, RS232, Ethernet, SD, JTAG, etc
Controller	Laptop	Laptop	Zynq SoC
Applicable Situation	Indoor	Indoor	Indoor
ROS Support	Yes	Yes	Yes (developing)
Scalability	Poor	Good	Excellent
Max. Speed (m/s)	0.65	1.2	1.5
Cost ($)	1995	>2000	1500

4 EXPERIMENTAL RESULTS

In the experimental part, a multi-robot following application is presented to demonstrate the feasibility of the proposed scheme. A lightweight robot node controlled by a Raspberry Pi board (robot node-2, shown in Figure 5) is added to constitute heterogeneous MRS with HRMRP (robot node-1). Robot node-2 has the same three layers as HRMRP and communicates with HRMRP using ROS TCP/IP protocol. The experimental procedure is as follows:

- Under the control of the server, robot node-1 explores the unknown environment and sends sensors information to the server. The SLAM is implemented to build a grid map by using an open-slam algorithm and visualized in RViz.
- When selecting a target location on map through RViz, the server calculates the global shortest path through Dijkstra algorithm and publishes the control commands to the robot nodes.

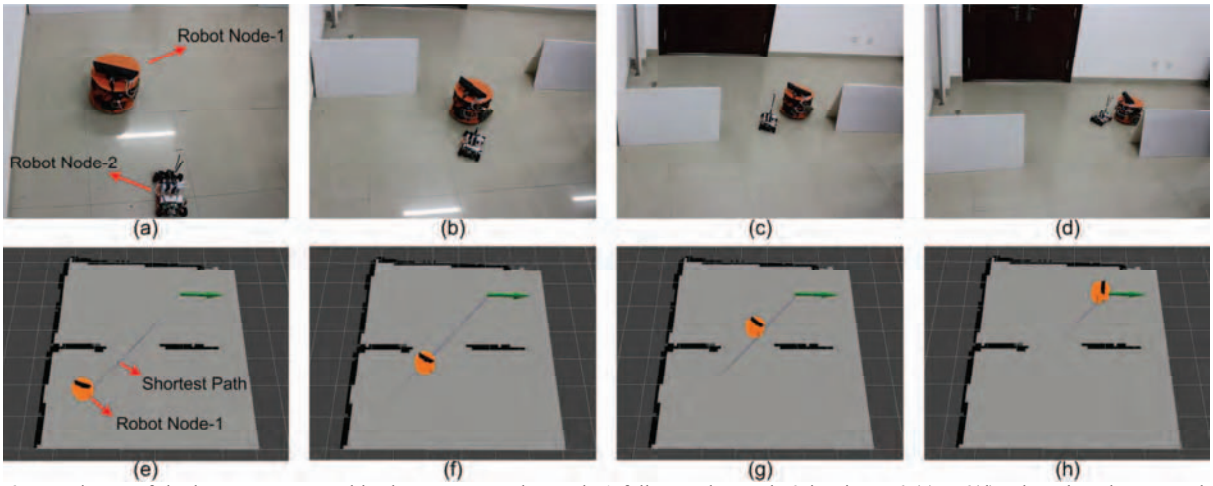

Fig. 6. Experiment of the heterogeneous multi-robot system. Robot node-1 follows robot node-2 in Figure 6 (a) – 6(d). The robot shortest path and real-time position are visualized on the map in Figure 6 (e) – 6(h).

- Robot node-1 moves forward along the path and makes appropriate adjustments based on obstacle location detected by its sensors. At the same time, the position and velocity are published to robot node-2.
- With the position and velocity, robot node-2 follows robot node-1 and reaches the destination.

The experimental result presented in Figure 6 is quite satisfactory. It demonstrates the feasibility and usability of the new ROS-based hybrid architecture proposed in this paper for heterogeneous MRS.

5 CONCLUSION

This paper presents a new ROS-based hybrid architecture for heterogeneous MRS, which is composed of PCs as the server and robots as the nodes. Compared with the existing traditional schemes, this architecture integrates PCs and embedded system together using ROS. The complex computation and visualization are processed on server, while real-time tasks are completed in robot nodes with embedded systems. In this way, the system can keep a good balance between flexibility, universality and expansibility, while the cost of MRS declines.

Further, we designed a low-cost and high-performance robot node named HRMRP. The robot node is realized on Zynq SoC. Other robot nodes with different structures can also be added into this network easily, such as TurtleBot-2, Pioneer 3-DX, etc. The heterogeneous MRS experimental result of multi-robot following is also presented to verify the robot system feasibility.

The future work will be focused on control and optimization of multi-robot cooperating work in the presented architecture. Also, more ROS function packages will be transplanted into embedded systems of robot nodes and be accelerated by FPGA.

REFERENCES

[1] A. Gautam, S. Mohan, A review of research in multi-robot systems, Proceedings of 7th International Conference on Industrial and Information Systems, 1-5, 2012.

[2] S. Han, M. S. Kim and H. S. Park, Open Software Platform for Robotic Services, IEEE Trans. Automation Science and Engineering, Vol.9, No.3, 467–481, 2012.

[3] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs E. Berger, R. Wheeler and A. Y. Ng, ROS: an open-source Robot Operating System, Proceedings of the IEEE International Conference on Robotics and Automation, 2009.

[4] T. Remmersmann, A. Tiderko; M. Langerwisch, S. Thamke, M. Ax, Commanding Multi-Robot Systems with Robot Operating System using Battle Management Language, Military Communications and Information Systems Conference, 1–6, 2012.

[5] Robot Operating System, http://www.ros.org/.

[6] S. Cousins, Exponential Growth of ROS, IEEE Robot. Autom. Mag., Vol.18, No.1, 19–20, 2011.

[7] S. Cousins, B. Gerkey, K. Conley, and W. Garage, Sharing Software with ROS [ROS Topics], IEEE Robot. Autom. Mag., Vol.17, No.2, 12–14, 2010.

[8] Y. Lei, Q. Zhu, Z. Feng, Research on architecture of multiple robots system based on the dynamic networks, Proceedings of the IEEE International Conference on Information and Automation, 93–98, 2010.

[9] Ardros, http://www.hessmer.org/blog/.

[10] Raspberry Pi, http://www.raspberrypi.org/.

[11] S. Zaman, W. Slany and G. Steinbauer, ROS-based mapping, localization and autonomous navigation using a Pioneer 3-DX robot and their relevant issues, Electronics. Communications and Photonics Conference, 1-5, 2011.

[12] W. Ren, D. X. He, J. Zhao, A Component Based Hybrid Embedded Framework for Mobile Robot, Proceedings of the IEEE International Conference on Parallel and Distributed Computing, 16-19, 2012.

[13] Zynq-7000 All Programmable SoC, http://www.xilinx.com/products/silicon-devices/soc/zynq-7000/index.htm.

[14] R. A. El-Laithy, J. Huang and M. Yeh, Study on the use of Microsoft Kinect for robotics applications, Position Location and Navigation Symposium, 1280–1288, 2012.

[15] TurtleBot 2, http://www.turtlebot.com/.

[16] Pioneer P3-DX, http://www.mobilerobots.com/Research-Robots/PioneerP3DX.aspx.